

Title

Filippo Rossi

Date

Contents

1	Introduzione	2
1.1	Alfabeti e parole	2
1.2	Algoritmo e decidibilit�	2
2	Automi a stati finiti	5
2.1	Definizione di automa a stati finiti.	5
2.2	Definizione di algoritmo associato ad un automa	5
3	Rappresentazione grafica degli automi	9
3.1	Automi come grafi	9
3.2	Matrici di adiacenza per grafi di automi finiti	10
4	Linguaggi regolari	13
5	Automi non deterministici	16
6	Espressioni regolari	25
7	Macchin di Turing	26
8	Teoria della complessit�	30
8.1	Look-up Table	31
9	Prerequisiti	34
9.1	Semianelli e serie formali	34

1 Introduzione

1.1 Alfabeti e parole

Definition 1.1 (Insieme delle parole su un alfabeto). Fissato un alfabeto A , si definisce l'insieme delle parole definite su A come l'insieme

$$A^* := \bigcup_{k \geq 0} A^{\{1, \dots, k\}}.$$

dove $A^{\{1, \dots, k\}} := \{f \text{ è una funzione} \mid f : \{1, \dots, k\} \rightarrow A\}$.

Si noti che per convenzione l'insieme $\{1, \dots, k\}$ per $k = 0$ corrisponde all'insieme vuoto. E pertanto l'insieme A^* per definizione contiene anche la funzione $A^\emptyset$ che è detta la *parola vuota*.

Definition 1.2 (Lunghezza di una parola). La *lunghezza* di una parola (in simboli $|u|$) è definita come il più piccolo n tale che $u \in A^{\{1, \dots, n\}}$.

Definition 1.3 (Concatenazione di parole). La *concatenazione di due parole* u e v è una funzione $uv \in A^{\{1, \dots, |u|+|v|\}}$ (indicata a volte anche come $u \cdot v$) definita come segue:

$$uv(k) := \begin{cases} u(k) & \text{se } k \leq |u| \\ v(k - |u|) & \text{se } |u| + 1 \leq k \leq |u| + |v| \end{cases}$$

Remark 1.4. Equivalentemente, possiamo introdurre l'insieme delle parole su A , con A^* , come l'insieme delle successioni finite di elementi di A (compresa la parola vuota). Se $u, v \in A^*$ allora $uv \in A^*$ indicherà la parola ottenuta concatenando u con v . Data l'equivalenza delle definizioni, useremo una piuttosto che l'altra in modo completamente arbitrario. Inoltre, visto che vale $A \hookrightarrow A^*$, considereremo A come un suo sottoinsieme a tutti gli effetti.

Remark 1.5. La tripla $(A^*, A^\emptyset, \cdot)$ è un monoide.

1.2 Algoritmo e decidibilità

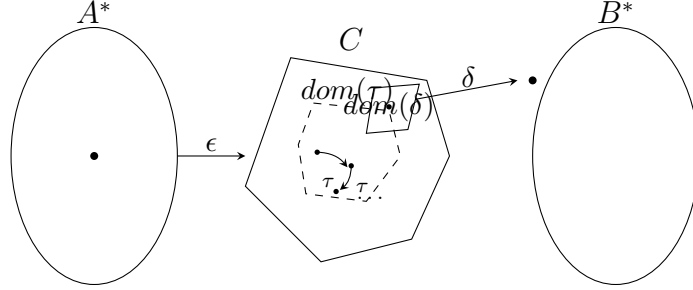
Definition 1.6 (Algoritmo). Un *algoritmo* è una sestupla $\mathcal{A} := (A, B, C, \epsilon, \tau, \delta)$ dove A e B sono insiemi chiamati rispettivamente *alfabeto di entrata* e *alfabeto di uscita*, C è un insieme chiamato *spazio di configurazioni possibili* e tre funzioni

$$\begin{aligned} \epsilon &: A^* \rightarrow C \\ \tau &: C \rightarrow C \\ \delta &: C \rightarrow B^* \end{aligned}$$

tali che $\{dom(\tau), dom(\delta)\}$ è una partizione di C . La *computazione associata ad una parola* $w \in A^*$ è la successione

$$(\tau^n(\epsilon(w)))_{n \in \mathbb{N}} = (\epsilon(w), \tau(\epsilon(w)), \tau^2(\epsilon(w)), \dots)$$

diremo che quest'ultima è *finita* (oppure diremo che *termina*) se esiste un $n \in \mathbb{N}$ tale che $\tau^n(\epsilon(w)) \in \text{dom}(\delta)$, altrimenti diremo che è *infinita*. Se la sequenza termina in una configurazione $\tau^n(\epsilon(w))$ allora avremo che il *risultato della computazione* è $\delta(\tau^n(\epsilon(w)))$. Se invece la computazione associata non termina si dirà che l'algoritmo *non termina partendo da* w .



Definition 1.7 (Funzione associata ad un algoritmo). Sia $\mathcal{A} := (A, B, C, \epsilon, \tau, \delta)$ un algoritmo. Possiamo definire la funzione

$$f_{\mathcal{A}}: A^* \rightarrow B^*$$

$$w \mapsto \begin{cases} \delta(\tau^n(\epsilon(w))) & \text{se } n \text{ è il più piccolo intero} \\ & \text{per cui } \tau^n(\epsilon(w)) \in \text{dom}(\delta), \\ \perp & \text{se } \mathcal{A} \text{ non termina partendo da } w \end{cases}$$

che chiameremo *funzione associata all'algoritmo* $\mathcal{A}$.

In pratica, ogni volta che diremo che la funzione f associata ad un certo algoritmo formale è definita per ogni possibile parola $w \in A^*$, intenderemo dire che per ogni possibile input la computazione termina e restituisce attraverso la funzione di output δ il valore della funzione $f(w)$.

Le due precedenti definizioni sono state date nel modo più generale possibile. D'ora in poi invece, a meno di casi particolari, considereremo solo algoritmi tali che $B = \{0, 1\}$.

Definition 1.8 (Algoritmo che decide un insieme). Diremo che un certo *algoritmo decide* un insieme $X \subset A^*$ quando la funzione associata all'algoritmo coincide su ogni elemento con la funzione caratteristica di X , quindi se vale:

$$f_{\mathcal{A}}(w) = \begin{cases} 1 & w \in X \\ 0 & w \notin X. \end{cases}$$

Remark 1.9. Se un algoritmo $\mathcal{A}$ decide un insieme X , allora abbiamo chiaramente che $f_{\mathcal{A}}$ è una funzione totale. In altre parole: quando un algoritmo decide un insieme, la computazione dell'algoritmo a partire da una qualsiasi parola di alfabeto A termina.

Definition 1.10 (Algoritmo che semidecide un insieme). Diremo che un certo algoritmo *semidecide* un insieme $X \subseteq A^*$ quando la funzione parziale associata all'algoritmo $f_{\mathcal{A}}$ è definita esattamente sugli elementi di X e, quando è definita, restituisce il valore 1. In altre parole:

$$f_{\mathcal{A}}(x) = \begin{cases} 1, & \text{se } x \in X, \\ \perp, & \text{se } x \notin X, \end{cases}$$

Chiaramente, uno dei due concetti e/' più forte dell'altro.

Proposition 1.11. Se esiste un algoritmo $\mathcal{A}$ che decide un iniseme $X \subset A^*$ su un alfabeto A ; allora esiste un algortmo $\mathcal{A}'$ che semidecide X .

Proof. Per ipotesi abbiamo un algoritmo $\mathcal{A} = (A, \{0, 1\}, C, \epsilon, \tau, \delta)$ che decide X . Possiamo considerare

$$T_0 := \delta^{-1}(\{0\}), \quad T_1 := \delta^{-1}(\{1\})$$

in modo tale da poter definire le seguenti funzioni:

$$\begin{aligned} \tau'(c) : C &\rightarrow C \\ c &\mapsto \begin{cases} c, & \text{se } c \in T_0 \\ \tau(c), & \text{se } c \notin T_0 \end{cases} \end{aligned}$$

$$\begin{aligned} \delta'(c) : C &\rightarrow \{0, 1\}^* \\ c &\mapsto \begin{cases} \delta(c), & \text{se } c \in T_1 \\ \perp, & \text{se } c \notin T_1 \end{cases} \end{aligned}$$

Possiamo definire quindi l'algoritmo $\mathcal{A} := (A, \{0, 1\}, C, \epsilon, \tau', \delta')$. Ora, per ogni $w \in A^*$ esiste un $k \in \mathbb{N}$ tale che $\tau^k(\epsilon(w)) \in \text{dom}(\delta) = T_0 \sqcup T_1$. Questo vuol dire che, per ogni $i < k$ vale che $\tau^i(\epsilon(w)) \notin T_0$. Quindi $\tau'^k(\epsilon(w)) = \tau^k(\epsilon(w))$. A questo punto dividiamo in casi:

$w \in X$: allora $\tau'^k(\epsilon(w)) \in T_1$, quindi $\delta'(\tau'^k(\epsilon(w))) = \delta(\tau'^k(\epsilon(w))) = 1$;

$w \notin X$: allora $\tau'^k(\epsilon(w)) \in T_0$, quindi $\delta'(\tau'^k(\epsilon(w))) = \perp$, entrando cosi/' in un ciclo.

Quindi, $\mathcal{A}'$ semidecide X .

□

Il viceversa non vale, vedremo un esempio più avanti **CONTROLLA**

controlla qui: Pertanto, consideriamo i processi computazionali come metodi di decisione simbolici, in cui si rende automatico (per questo formalmente ineccepibile) il metodo di decisione di una proprietà. Le garanzie sulla correttezza della decisione vengono dalla verificabilità: il processo lavora su sequenze di simboli, e applica in maniera asettica alcune regole di trasformazione (i principi deduttivi) in maniera puntuale: da cui viene la richiesta specifica che le regole di trasformazione siano locali, applicabili in tempo finito e soprattutto in modo che la verifica della corretta applicazione sia finita.

In questo quadro, se anche il problema di decisione riguardasse un insieme di cardinalità più che numerabile, un processo computazionale finito potrebbe procedere modificando ed essendo influenzato solo da un numero finito di elementi. Quindi è la descrizione stessa del problema che ha senso solo quando è finita.

Con queste premesse la classe dei problemi più semplici che si possono porre è in prima istanza quella dei *problemi di decisione* per un fissato alfabeto $A = \{a_1, \dots, a_n\}$. Essi consistono nella possibilità di definire un metodo per stabilire la separazione tra le parole di un sottoinsieme $X \subset A^*$ e quelle del complemento $A^* \setminus X$, procedura con le caratteristiche che abbiamo ricordato.

2 Automi a stati finiti

2.1 Definizione di automa a stati finiti.

Definition 2.1. Siano A e Q insiemi finiti, un *automa finito* di alfabeto A e insieme degli stati Q è una tripla (T, q_0, F) dove

$$T : Q \times A \rightarrow Q$$

$F \subset Q$ e $q_0 \in Q$ è uno stato particolare, detto stato iniziale.

2.2 Definizione di algoritmo associato ad un automa

A partire dalla tripla (T, q_0, F) che specifica un dato automa di alfabeto A e insieme degli stati Q si costruisce prima l'insieme delle configurazioni possibili:

Definition 2.2 (Configurazione per Automa Finito). Una configurazione per automa a stati finiti di alfabeto A e insieme degli stati Q è una tripla (w, p, q) , dove

1. $w \in A^*$ è la parola da analizzare,
2. $p \in \mathbb{N}$, è un intero che indica la posizione della parola da analizzare nella prossima transizione,
3. $q \in Q$ indica lo stato corrente e costituisce la memoria delle transizioni effettuate nei precedenti passi di computazione.

L'insieme delle configurazioni di alfabeto A e insieme degli stati Q per automa finito sarà denotato con

$$Conf_{DFA}^{A,Q} := A^* \times \mathbb{N} \times Q$$

ed allora possiamo definire:

Definition 2.3 (Algoritmo associato ad un automa finito deterministico). L'algoritmo associato ad un automa di alfabeto A e insieme degli stati Q è l'algoritmo

$$(A, Conf_{DFA}^{A,Q}, \{0, 1\}, \epsilon, \tau, \delta)$$

dove le tre funzioni sono definite come segue:

- la **funzione di ingresso** che associa ad una parola $w \in A^*$, la configurazione $\epsilon(w) = (w, 1, q_0)$,
- la **funzione di transizione** $\tau : Conf_{DFA}^{A,Q} \rightarrow Conf_{DFA}^{A,Q}$ è data da

$$\tau((w, p, q)) = (w, p + 1, T(q, w(p))),$$

- la **funzione di uscita** $\delta : Conf_{DFA}^{A,Q} \rightarrow \{0, 1\}^*$, data da

$$\delta((w, p, q)) = \begin{cases} 0 & \text{se } p > |w| \text{ e se } q \notin F, \\ 1 & \text{se } p > |w| \text{ e se } q \in F. \end{cases}$$

Remark 2.4. L'idea dietro la definizione precedente e/' quella di leggere una lettera alla volta della parola w e tramite la funzione T cambiare lo stato della macchina in base alla lettera letta e allo stato corrente della macchina. Ovviamente la computazione finisce quando sono state esaminate tutte le lettere della parola (che vengono esaminate esattamente una volta sola). Inoltre, un automa non ha una vera e propria memoria, bensì/' ha gli stati (che giocano il ruolo della memoria dell'automa).

Definition 2.5 (Insiemi decidibili per automa). Un insieme $X \subset A^*$, è *decidibile* per automa finito (oppure *DFA-decidibile* oppure *regolare*), se esiste un insieme finito Q e un automa finito con alfabeto A e insieme degli stati Q tale che l'algoritmo associato decide X .

Example 2.6. Vogliamo ora descrivere l'automa finito che decide l'insieme delle parole di lunghezza pari (risp. dispari) su un alfabeto dato A . A partire dall'alfabeto finito A , e da un insieme di due stati $Q = \{0, 1\}$ che corrisponderanno rispettivamente allo stato "abbiamo letto un numero pari (oppure dispari) di caratteri della parola"; costruiamo la funzione di transizione che ad ogni carattere letto cambia stato:

$$T : Q \times A \rightarrow Q$$

tale che

$$T(q, a) = \begin{cases} 1 & \text{se } q = 0 \\ 0 & \text{se } q = 1, \end{cases}$$

per ogni $a \in A$. Avremo allora che lo stato iniziale $q_0 = 0$, poiché inizialmente non avremo ancora letto nessun carattere e che l'insieme degli stati accettanti sarà: $F = \{0\}$. Sia quindi

$$X = \{ w \in A^* : |w| \text{ è pari} \}.$$

e dimostriamo che l'algoritmo associato all'automa

$$(T, q_0, F)$$

decide l'insieme X . Sia dunque $w = a_1 a_2 \cdots a_n \in A^*$. La configurazione iniziale è

$$\epsilon(w) = (w, 1, 0).$$

Ad ogni applicazione della funzione di transizione viene letta una lettera della parola e lo stato viene invertito:

$$0 \longleftrightarrow 1.$$

Per induzione sul numero di transizioni effettuate si ottiene che dopo aver letto i primi k caratteri della parola la configurazione è

$$(w, k + 1, q_k),$$

dove

$$q_k = \begin{cases} 0 & \text{se } k \text{ è pari,} \\ 1 & \text{se } k \text{ è dispari.} \end{cases}$$

Dopo la lettura dell'intera parola si raggiunge quindi la configurazione

$$(w, n + 1, q_n).$$

Poiché

$$n + 1 > |w|,$$

questa appartiene al dominio della funzione di uscita. Inoltre

$$q_n = \begin{cases} 0 & \text{se } |w| \text{ è pari,} \\ 1 & \text{se } |w| \text{ è dispari.} \end{cases}$$

Essendo $F = \{0\}$, segue che

$$\delta(w, n + 1, q_n) = \begin{cases} 1 & \text{se } |w| \text{ è pari,} \\ 0 & \text{se } |w| \text{ è dispari.} \end{cases}$$

Pertanto

$$f_A(w) = \begin{cases} 1 & \text{se } w \in X, \\ 0 & \text{se } w \notin X, \end{cases}$$

cioè

$$f_A = \mathbf{1}_X.$$

Ne consegue che l'algoritmo associato all'automa decide l'insieme delle parole di lunghezza pari.

Exercise 1. Fissiamo l'alfabeto binario $A = \{0, 1\}$, dimostrare che l'insieme

$$X = \{w0 \mid w \in A^*\}$$

è decidibile per automa a stati finiti.

Soluzione. Si richiede di mostrare un automa il cui algoritmo associato accetta le parole che terminano con uno zero. È dunque sufficiente memorizzare nello stato l'ultimo simbolo letto e alla fine della lettura della parola decidere in base allo stato: poniamo $Q = \{q_0, q_1\}$, e definiamo

$$T(q, a) := \begin{cases} q_0 & \text{se } a = 0, \\ q_1 & \text{se } a = 1, \end{cases}$$

prendiamo q_0 come stato iniziale e $F = \{q_0\}$ come insieme degli stati finali accettanti.

Proof. Sia

$$X = \{w0 \mid w \in A^*\}$$

l'insieme delle parole sull'alfabeto binario che terminano con il simbolo 0.

Costruiamo un automa finito che decide X . Consideriamo l'insieme degli stati

$$Q = \{q_0, q_1\},$$

dove intuitivamente:

- q_0 rappresenta la situazione in cui l'ultimo simbolo letto è 0;
- q_1 rappresenta la situazione in cui l'ultimo simbolo letto è 1.

Definiamo la funzione di transizione

$$T : Q \times A \rightarrow Q$$

come

$$T(q, a) = \begin{cases} q_0 & \text{se } a = 0, \\ q_1 & \text{se } a = 1. \end{cases}$$

Scegliamo come stato iniziale

$$q_{\text{init}} = q_0$$

e come insieme degli stati accettanti

$$F = \{q_0\}.$$

Consideriamo ora l'algoritmo associato all'automa. Sia

$$w = a_1 a_2 \cdots a_n \in A^*.$$

La configurazione iniziale è

$$\epsilon(w) = (w, 1, q_0).$$

Dopo k transizioni, per induzione su k , la configurazione raggiunta è

$$(w, k + 1, q_k),$$

dove

$$q_k = \begin{cases} q_0 & \text{se } a_k = 0, \\ q_1 & \text{se } a_k = 1. \end{cases}$$

Infatti, ad ogni passo la funzione di transizione sostituisce lo stato corrente con lo stato corrispondente all'ultimo carattere letto.

Dopo aver letto tutta la parola w , la computazione termina nella configurazione

$$(w, n + 1, q_n).$$

La funzione di uscita restituisce quindi

$$\delta(w, n + 1, q_n) = \begin{cases} 1 & \text{se } q_n \in F, \\ 0 & \text{se } q_n \notin F. \end{cases}$$

Poiché

$$F = \{q_0\},$$

si ha

$$\delta(w, n + 1, q_n) = \begin{cases} 1 & \text{se l'ultimo simbolo di } w \text{ è } 0, \\ 0 & \text{se l'ultimo simbolo di } w \text{ è } 1. \end{cases}$$

Quindi

$$f_A(w) = \begin{cases} 1 & \text{se } w \in X, \\ 0 & \text{se } w \notin X. \end{cases}$$

Pertanto

$$f_A = \mathbf{1}_X$$

e quindi l'algoritmo associato all'automa decide X .

Concludiamo che

$$X = \{w0 \mid w \in A^*\}$$

è DFA-decidibile. □

3 Rappresentazione grafica degli automi

3.1 Automi come grafi

A partire da un automa finito (T, q_0, F) se ne può dare una descrizione grafica nel modo seguente: si consideri il grafo i cui nodi sono gli stati Q , e gli archi etichettati con elementi di A che collegano due stati (q_i, q_j) se esiste la transizione $T(q_i, a) = q_j$. inoltre avremo una funzione che ci indica lo stato iniziale

$$\text{init}(G_{(T, q_0, F)}) = q_0$$

(di solito indicata da un arco entrante nel nodo iniziale q_0) ed una che distingue gli stati accettanti (risp. non accettanti)

$$\text{acc}(G_{(T, q_0, F)}) = F \quad (\text{risp. } \text{ref}(G_{(T, q_0, F)}) = Q \setminus F)$$

(di solito si indicano gli stati accettanti con un doppio cerchio, mentre tutti gli altri nodi sono considerati non accettanti). Per essere più formali:

Definition 3.1 (Grafo associato ad un automa). Dato un automa (T, q_0, F) definito su un alfabeto A e insieme di stati Q definiamo il suo grafo associato $G = (V, E, L)$ come segue:

- l'insieme dei vertici è

$$V := Q;$$

- l'insieme dei lati è

$$E := \{(q_i, a, q_j) \in Q \times A \times Q \mid T(q_i, a) = q_j\};$$

- la funzione di etichettatura è

$$L : E \rightarrow A, \quad L(q_i, a, q_j) = a.$$

Ad esempio il grafo associato all'automa dell'Esempio 2.6, è rappresentato in Figura 1.

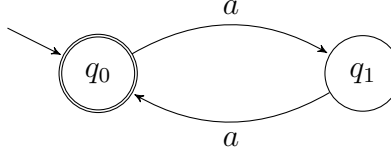


Figure 1: Automa dell'Esempio 2.6, notare che a è un qualsiasi elemento nell'alfabeto $A = \{0, 1\}$.

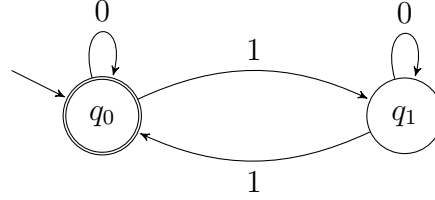


Figure 2: In modo simile all'automa dell'Esempio 2.6 si può dare l'automa che decide l'insieme di tutte le parole con un numero pari di 1.

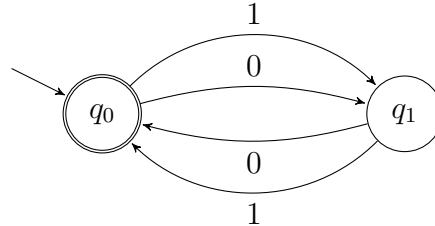
3.2 Matrici di adiacenza per grafi di automi finiti

Consideriamo un automa (T, q_0, F) definito su un alafabeto A e insieme di stati Q . Possiamo sempre considerare il monoide $M = (A^*, A^\emptyset, \cdot)$ e l'anello (che e/' anche un semianello) $\mathbb{N}$. A questo punto possiamo definire l'insieme delle serie formali

$$\mathbb{N}[M] := \left\{ \sum_{w \in A^*} n_w w \mid n_w \in \mathbb{N} \right\}$$

Ora possiamo considerare la matrice di adiacenza Z dell'automa, che non sarà nient'altro che una matrice cha nelle entrate elementi di $\mathbb{N}[M]$. Come sappiamo, fare la potenza k -esima di una matrice di adiacenza con etichette, fornisce nell'entrata (i, j) tutti e i soli cammini di lunghezza k che collegano il vertice i al vertice j (nel caso degli automi, dallo stato q_i allo stato q_j). Vediamo un esempio:

Example 3.2. Consideriamo l'automa che decide l'inisieme delle parole di lunghezza pari. Questo e/' rappresentato dal seguente grafo etichettato.



La sua matrice di adiacenza e/'

$$Z = \begin{pmatrix} \perp & 0 + 1 \\ 0 + 1 & \perp \end{pmatrix}$$

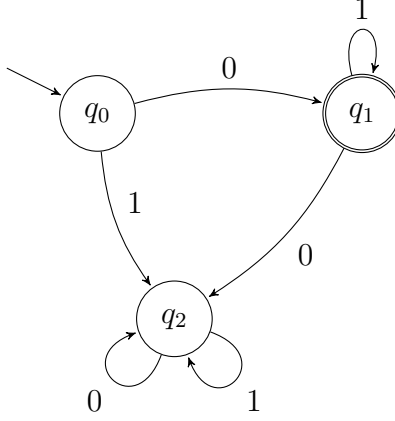


Figure 3: Automa che decide l'insieme $X = \{01^n \mid n \geq 0\}$.

Infatti, gli archi 0 e 1 collegano i vertici q_0 e q_1 in entrambe le direzioni. Inoltre, visto che non ci sono loop, sulla diagonale si inserisce convenzionalmente la serie formale vuota, indicata con $\perp$. A questo punto vogliamo trovare i cammini di lunghezza 2 del grafo. Per farlo basta calcolare la potenza seconda di Z :

$$\begin{aligned}
 Z^2 &= \begin{pmatrix} \perp\perp + (0+1)(0+1) & \perp(0+1) + (0+1)\perp \\ (0+1)\perp + \perp(0+1) & (0+1)(0+1) + \perp\perp \end{pmatrix} \\
 &= \begin{pmatrix} 00 + 01 + 10 + 11 & \perp + \perp \\ \perp + \perp & 00 + 01 + 10 + 11 \end{pmatrix} \\
 &= \begin{pmatrix} 00 + 01 + 10 + 11 & \perp \\ \perp & 00 + 01 + 10 + 11 \end{pmatrix}
 \end{aligned}$$

Per verificare le operazioni effettuate con $\perp$, basta usare la definizione presente in (9.15). Dalla matrice possiamo quindi dedurre che esistono 4 cammini che collegano q_0 e q_1 con loro stessi, e nessuno che collega q_0 con q_1 o viceversa.

A questo punto ci vogliamo porre la seguente domanda: si può dedurre dalla matrice di adiacenza tutti i cammini che uniscono lo stato iniziale ad uno stato accettante dell'automa? In altre parole: dalla matrice di adiacenza si può dedurre il calcolo che l'automa compie per decidere un insieme? La risposta è sì. In particolare, è anche possibile dedurre l'intero insieme X da decidere. Prima di affrontare il caso generale analizziamo questo problema nel caso dell'esempio precedente.

Example 3.3. In questo caso esiste un solo stato accettante, che coincide esattamente con lo stato iniziale. Quindi, in una qualsiasi potenza di Z , dovremmo considerare l'entrata $(1,1)$ per trovare le parole accettate dall'automa. Più precisamente, per ogni k naturale, nell'entrata $(1,1)$ della matrice Z^k troviamo le uniche e sole parole di lunghezza k accettate dall'automa.

Lavoriamo ora al caso generale. Se con Z indichiamo la matrice di adiacenza di un

automa, possiamo definire definiamo

$$E(Z) := \sum_{k=0}^{\infty} Z^k$$

dove Z^0 e/' la matrice cha ha sulla diagonale la parola vuota $A^\emptyset$ e nelle altre entrate la serie vuota $\perp$. Assumendo che lo stato iniziale sia q_0 e che corrisponda alla prima riga della matrice Z , allora sulla prima riga di $E(Z)$ troveremo le parole che l'automa accetta nelle colonne corrispondenti agli stati accettanti.

Example 3.4. Facendo ancora un paio di potenze di Z , si può dedurre la seguente formula:

$$Z^k = \begin{cases} \begin{pmatrix} (0+1)^k & \perp \\ \perp & (0+1)^k \end{pmatrix}, & \text{se } k \text{ e/' pari} \\ \begin{pmatrix} \perp & (0+1)^k \\ (0+1)^k & \perp \end{pmatrix}, & \text{se } k \text{ e/' dispari} \end{cases}$$

quindi abbiamo

$$\begin{aligned} E(Z) &= \begin{pmatrix} A^\emptyset & \perp \\ \perp & A^\emptyset \end{pmatrix} + \begin{pmatrix} \perp & (0+1) \\ (0+1) & \perp \end{pmatrix} + \begin{pmatrix} (0+1)^2 & \perp \\ \perp & (0+1)^2 \end{pmatrix} + \cdots \\ &= \begin{pmatrix} A^\emptyset + \sum_{k=1}^{\infty} (0+1)^{2k} & \perp + \sum_{k=0}^{\infty} (0+1)^{2k+1} \\ \perp + \sum_{k=0}^{\infty} (0+1)^{2k+1} & A^\emptyset + \sum_{k=1}^{\infty} (0+1)^{2k} \end{pmatrix} \\ &= \begin{pmatrix} A^\emptyset + \sum_{k=1}^{\infty} (0+1)^{2k} & \sum_{k=0}^{\infty} (0+1)^{2k+1} \\ \sum_{k=0}^{\infty} (0+1)^{2k+1} & A^\emptyset + \sum_{k=1}^{\infty} (0+1)^{2k} \end{pmatrix} \end{aligned}$$

Per quanto osservato prima, nel posto (1,1) della matrice $E(Z)$ troviamo le uniche e solo parole accettate dall'automa. **Non coverrebbe scrivere E(Z) a partire con k=1? perhce ha priori non e/' vero che la parola vuota e/' decidibile**

Usando un po' di algebra lineare, e/' possibile esplicitare esattamente l'inisieme delle parole accettate. Infatti, fi possiamo definire i seguenti vettori:

$$\begin{aligned} V_0(i) &= \begin{cases} A^\emptyset, & q_i = q_0 \\ \perp, & q_i \neq q_0 \end{cases} \\ V_F(i) &= \begin{cases} A^\emptyset, & q_i \in F \\ \perp, & q_i \notin F \end{cases} \end{aligned}$$

E calcolando il prodotto righe per colonne

$$V_0 \cdot E(Z) \cdot V_F^t$$

otteniamo la serie formale che ha come addendi esattamente le parole accettate da X . La formula appena enunciata prende il nome di *formula dell'esecuzione*.¹

¹Ricordiamo che $\perp$ e/' l'elemento neutro additivo, che $A^\emptyset$ e/' l'elemento neutro moltiplicativo, e che $\perp \cdot w = \perp$ per ogni serie formale $w \in \mathbb{N}[M]$.

Example 3.5. Nel caso dell'esempio precedente abbiamo

$$V_0 = (A^\emptyset, \perp)$$

$$V_F = (A^\emptyset, \perp)$$

Facendo il prodotto abbiamo

$$\begin{aligned} V_0 \cdot E(Z) \cdot V_F^t &= (A^\emptyset \quad \perp) \cdot \begin{pmatrix} A^\emptyset + \sum_{k=1}^{\infty} (0+1)^{2k} & \sum_{k=0}^{\infty} (0+1)^{2k+1} \\ \sum_{k=0}^{\infty} (0+1)^{2k+1} & A^\emptyset + \sum_{k=1}^{\infty} (0+1)^{2k} \end{pmatrix} \cdot \begin{pmatrix} A^\emptyset \\ \perp \end{pmatrix} \\ &= (A^\emptyset + \sum_{k=1}^{\infty} (0+1)^{2k} \quad \sum_{k=0}^{\infty} (0+1)^{2k+1}) \cdot \begin{pmatrix} A^\emptyset \\ \perp \end{pmatrix} \\ &= A^\emptyset + \sum_{k=1}^{\infty} (0+1)^{2k} \end{aligned}$$

Che e/' esattamente quanto abbiamo osservato precedentemente.

4 Linguaggi regolari

Per determinare se un determinato linguaggio è regolare: mentre la dimostrazione sufficiente di AF- decidibilità (esibendo esplicitamente un automa) può risultare noiosa e complicata, è possibile fornire dimo- strazioni alternative (possibilmente più semplici) mostrando che l'insieme è ottenibile a partire da insiemi AF-decidibili utilizzando operazioni "compatibili" con l'AF-decidibilità (proprietà di chiusura). In questa sezione vedremo dunque che esistono operazioni fatte su insiemi AF-decidibili che permettono di ottenere un altro insieme DFA-decidibile.

Definition 4.1. Sia A un alfabeto e X_1 e x_2 due inisemi di parole su A , ovvero $X_1, X_2 \subset A^*$. Definiamo allora:

- l'operazione di *concatenazione*:

$$X_1 \cdot X_2 := \{w_1 w_2 \in A^* \mid w_i \in X_i\};$$

- l'operazione di *somma*:

$$X_1 + X_2 := \{w \in A^* \mid w \in X_1 \text{ oppure } w \in X_2\}$$

(di fatto, è l'unione insiemistica di X_1 con X_2);

- l'operazione di *star di Kleene*:

$$X^* := \{w_1 w_2 \dots w_n \mid w_i \in X, \text{ for } i = 1, \dots, n \text{ e } n \geq 0\}$$

Proposition 4.2. Siano X_1 e X_2 regolari allora anche $X_1 + X_2$ è regolare.

Proof. Sia $\mathcal{A}_1 = (T_1, q_0, F_1)$ l'automa che decide l'insieme X_1 e sia $\mathcal{A}_2 = (T_2, q'_0, F_2)$ l'automa che decide X_2 . L'automa che riconosce $X_1 + X_2$ è ottenuto utilizzando il prodotto cartesiano degli stati dei due automi e calcolando l'evoluzione “parallela” di entrambi gli automi a partire da uno stato coppia, in questo modo se una delle due componenti dello stato è accettante alla fine della computazione la parola sarà accettata poiché apparterrà ad uno dei due insiemi e quindi sarà nella loro unione.

Costruzione dell'automa $\mathcal{A} = (T, q''_0, F)$ di alfabeto A e insieme degli stati $Q = Q_1 \times Q_2$: definiamo

$$q''_0 = (q_0, q'_0), \quad F = F_1 \times Q_2 \cup Q_1 \times F_2$$

e

$$T((q_1, q_2), a) = (T_1(q_1, a), T_2(q_2, a)).$$

Mostriamo che $\mathcal{A}$ decide precisamente $X_1 + X_2$.

$$T^*((q_0, q'_0), w) = (T_1^*(q_0, w), T_2^*(q'_0, w)).$$

Mostriamo ora che l'automa $\mathcal{A}$ decide l'insieme $X_1 + X_2$, ovvero che la funzione associata all'automa coincide con la funzione caratteristica di tale insieme.

Sia $w \in A^*$. Dalla definizione dell'insieme degli stati finali si ha:

$$\begin{aligned} f_{\mathcal{A}}(w) = 1 &\iff T^*((q_0, q'_0), w) \in F \\ &\iff (T_1^*(q_0, w), T_2^*(q'_0, w)) \in (F_1 \times Q_2) \cup (Q_1 \times F_2) \\ &\iff T_1^*(q_0, w) \in F_1 \text{ oppure } T_2^*(q'_0, w) \in F_2 \\ &\iff f_{\mathcal{A}_1}(w) = 1 \text{ oppure } f_{\mathcal{A}_2}(w) = 1 \\ &\iff w \in X_1 \text{ oppure } w \in X_2 \\ &\iff w \in X_1 + X_2. \end{aligned}$$

Quindi, per ogni $w \in A^*$,

$$f_{\mathcal{A}}(w) = \mathbf{1}_{X_1 + X_2}(w).$$

Pertanto l'automa $\mathcal{A}$ decide l'insieme $X_1 + X_2$, e quindi $X_1 + X_2$ è regolare. $\square$

Exercise 2. Si consideri l'alfabeto $A = \{0, 1\}$ e sia $B = \{1\}$ la restrizione all'insieme che contiene il solo simbolo 1; si considerino l'insieme delle parole di alfabeto A di lunghezza pari:

$$X_1 = \{w \in A^* \mid |w| = 2n, n \geq 0\}$$

e l'insieme delle parole di alfabeto A che contengono un numero pari di 1:

$$X_2 = \{w \in A^* \mid |w|_B = 2k, k \geq 0\}$$

qui si è utilizzata la notazione $w|_B$ per indicare la parola w ristretta ai soli elementi di alfabeto B , (si noti che si tratta degli insiemi considerati in Figura ?? e in Figura 2).

Costruire l'automa che decide $X_1 + X_2$.

Proof. L'automa si può ottenere utilizzando la costruzione di composizione parallela utilizzata nella prova della proposizione a partire dai due automi per X_1 e X_2 . $\square$

Mostriamo ora un'altra proprietà di chiusura degli insiemi regolari: la classe degli insiemi regolari è chiusa per concatenazione, ovvero:

Proposition 4.3. Siano X_1 e X_2 regolari allora anche $X_1 \cdot X_2$ è regolare.

Potremmo tentare la costruzione di un automa come nella proposizione precedente in modo che ogni parola $w \in X_1X_2$ abbia un inizio riconosciuto dall'automa che riconosce X_1 e continui con una seconda parte che sia accettata dall'automa che riconosce X_2 . In pratica si potrebbe concatenare il primo automa con il secondo, ma il problema sarebbe che potrebbero esserci più modi di fattorizzare w in modo che il primo automa accetti la prima parte di w . Per poter definire l'automa concatenazione bisognerebbe sapere a quale punto passare dal primo al secondo automa (e non è detto che ci sia un unico punto che vada bene per tutte le parole di $X_1 \cdot X_2$). Per risolvere questo problema si introduce una nuova nozione: la nozione di computazione *non-deterministica*.

5 Automi non deterministici

Il concetto di computazione non deterministica viene introdotto per modellare la possibilità di effettuare transizioni multiple (in questo caso il non determinismo modella una sorta di computazione parallela) oppure la possibilità di scegliere tra più possibili evoluzioni della computazione. Per ottenere una versione non deterministica del concetto di automa a stati finiti introdotto nella sezione precedente, bisogna variare la definizione nel punto in cui si stabilisce che Q è il codominio della tabella delle transizioni, sostituendo Q con l'insieme delle parti di Q , ovvero 2^Q : quindi dando alla funzione T la possibilità di fornire non un unico stato ma un insieme di stati; in questo modo avremo un modello di calcolo simile al precedente con una dinamica non deterministica poiché può seguire più computazioni allo stesso tempo, quindi definiamo:

Definition 5.1 (Automa Finito Non-deterministico). Siano A e Q insiemi finiti. Un *automa finito non-deterministico* di alfabeto A e insieme degli stati Q è una tripla (T, q_0, F) dove

$$T : Q \times A \longrightarrow 2^Q,$$

$$F \subseteq Q,$$

e $q_0 \in Q$ è uno stato particolare, detto *stato iniziale*.

Come nel caso deterministico, associamo alla nozione di automa finito non-deterministico quella di algoritmo formale che fornisce una semantica (operazionale) al modello di calcolo, mostrando come la computazione viene effettuata dall'automa. Per prima cosa è necessario stabilire lo spazio delle configurazioni.

Definition 5.2 (Configurazione per Automa Finito Non-deterministico). Una configurazione per un automa finito non-deterministico di alfabeto A e insieme degli stati Q è una tripla (w, p, S) , dove

1. $w \in A^*$ è la parola da analizzare;
2. $p \in \mathbb{N}$ è il puntatore alla posizione della parola che contiene il simbolo dell'alfabeto da considerare per effettuare la successiva transizione;
3. $S \subseteq 2^Q$ è un insieme di stati.

L'insieme delle configurazioni di alfabeto A e insieme degli stati Q per un automa finito sarà denotato con

$$\text{Conf}_{ND}^{A,Q} = A^* \times \mathbb{N} \times 2^Q.$$

Inoltre introduciamo la seguente notazione, quando $S \subseteq Q$:

$$T(S, a) := \bigcup_{q \in S} \{T(q, a)\}$$

Grazie a tale notazione possiamo definire l'algoritmo associato ad un automa finito non-deterministico.

Definition 5.3 (Algoritmo associato ad un Automa Finito Non-deterministico). L'algoritmo associato ad un automa finito non-deterministico di alfabeto A e insieme degli stati Q è la tupla $(A, \{0, 1\}, Conf_{ND}^{A,Q}, \varepsilon, \tau, \delta)$ dove le tre funzioni si definiscono come segue :

- la *funzione di ingresso*

$$e : A^* \longrightarrow Conf_{ND}^{A,Q},$$

definita da

$$e(w) = (w, 1, \{q_0\});$$

- la *funzione di transizione*

$$\tau : Conf_{ND}^{A,Q} \longrightarrow Conf_{ND}^{A,Q},$$

definita da

$$\tau((w, p, S)) = (w, p + 1, T(S, w(p)));$$

- la *funzione di uscita*

$$\delta : Conf_{ND}^{A,Q} \longrightarrow \{0, 1\}^*,$$

definita da

$$\delta((w, p, S)) = \max_{q \in S} F'(q), \quad \text{se } p = |w| + 1.$$

Di conseguenza resta definito il concetto di decidibilità di un insieme mediante automi finiti non-deterministici.

Definition 5.4 (Insieme decidibile per Automa Finito Non-deterministico). Un insieme $X \subseteq A^*$ è detto *decidibile per automa finito non-deterministico* (o *NDFA-decidibile*) se esistono un insieme finito Q ed un automa finito non-deterministico di alfabeto A e insieme degli stati Q tali che l'algoritmo associato decide X .

Un primo esempio di automa finito non-deterministico:

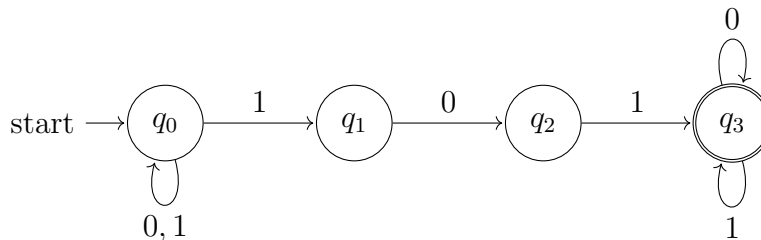
Example 5.5 (Un esempio di NDFA). Consideriamo $A = \{0, 1\}$, $Q = \{q_0, q_1, q_2, q_3\}$ e definiamo il seguente NDFA

$$(T, q_0, F)$$

dove $F := \{q_3\}$ e

$$\begin{array}{lll} T : Q \times \{0, 1\} & \rightarrow & 2^Q \\ (q_0, 0) & \mapsto & \{q_0\} \\ (q_0, 1) & \mapsto & \{q_0, q_1\} \\ (q_1, 0) & \mapsto & \{q_2\} \\ (q_1, 1) & \mapsto & \emptyset \\ (q_2, 0) & \mapsto & \emptyset \\ (q_2, 1) & \mapsto & \{q_3\} \\ (q_3, 0) & \mapsto & \{q_3\} \\ (q_3, 1) & \mapsto & \{q_3\} \end{array}$$

che può essere rappresenato graficamente come



Analizziamo ora il comportamento dell'automa su alcune parole:

- Consideriamo la parola

$$w = 1101.$$

Calcoliamo l'insieme degli stati raggiungibili dopo la lettura di ciascun simbolo:

$$\begin{aligned}\hat{T}(q_0, 1) &= \{q_0, q_1\}, \\ \hat{T}(q_0, 11) &= T(q_0, 1) \cup T(q_1, 1) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}, \\ \hat{T}(q_0, 110) &= T(q_0, 0) \cup T(q_1, 0) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}, \\ \hat{T}(q_0, 1101) &= T(q_0, 1) \cup T(q_2, 1) = \{q_0, q_1\} \cup \{q_3\} = \{q_0, q_1, q_3\}.\end{aligned}$$

Poiché l'insieme finale contiene lo stato finale q_3 , la parola

$$1101$$

è accettata.

- Consideriamo ora la parola

$$w = 1000.$$

Si ha

$$\begin{aligned}\hat{T}(q_0, 1) &= \{q_0, q_1\}, \\ \hat{T}(q_0, 10) &= T(q_0, 0) \cup T(q_1, 0) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}, \\ \hat{T}(q_0, 100) &= T(q_0, 0) \cup T(q_2, 0) = \{q_0\} \cup \emptyset = \{q_0\}, \\ \hat{T}(q_0, 1000) &= T(q_0, 0) = \{q_0\}.\end{aligned}$$

L'insieme degli stati raggiungibili non contiene alcuno stato finale, quindi la parola

$$1000$$

è rifiutata.

- Consideriamo infine la parola

$$w = 1100101.$$

Si ha

$$\begin{aligned}
\hat{T}(q_0, 1) &= \{q_0, q_1\}, \\
\hat{T}(q_0, 11) &= T(q_0, 1) \cup T(q_1, 1) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}, \\
\hat{T}(q_0, 110) &= T(q_0, 0) \cup T(q_1, 0) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}, \\
\hat{T}(q_0, 1100) &= T(q_0, 0) \cup T(q_2, 0) = \{q_0\} \cup \emptyset = \{q_0\}, \\
\hat{T}(q_0, 11001) &= T(q_0, 1) = \{q_0, q_1\}, \\
\hat{T}(q_0, 110010) &= T(q_0, 0) \cup T(q_1, 0) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}, \\
\hat{T}(q_0, 1100101) &= T(q_0, 1) \cup T(q_2, 1) = \{q_0, q_1\} \cup \{q_3\} = \{q_0, q_1, q_3\}.
\end{aligned}$$

Poiché l'insieme finale contiene lo stato finale q_3 , la parola

$$1100101$$

è accettata.

Si può mostrare infatti che questo automa è quello che decide l'insieme delle parole che contengono come sottostringa 101. Consideriamo ora lo stesso automa sostituendo la transizione

$$T(q_0, 1) = \{q_0, q_1\}$$

con

$$T(q_0, 1) = \{q_1\},$$

lasciando invariate tutte le altre transizioni. Mostriamo che questo automa non riconosce più tutte le parole contenenti la sottostringa 101. Infatti, la parola

$$w = 1101$$

contiene la sottostringa 101, ma l'automa si comporta nel modo seguente:

$$\begin{aligned}
\hat{T}(q_0, \varepsilon) &= \{q_0\}, \\
\hat{T}(q_0, 1) &= \{q_1\}, \\
\hat{T}(q_0, 11) &= T(q_1, 1) = \emptyset, \\
\hat{T}(q_0, 110) &= \bigcup_{q \in \emptyset} T(q, 0) = \emptyset, \\
\hat{T}(q_0, 1101) &= \bigcup_{q \in \emptyset} T(q, 1) = \emptyset.
\end{aligned}$$

Poiché la computazione termina con l'insieme vuoto, la parola 1101 viene rifiutata.

L'errore consiste nel fatto che l'automa è costretto a scegliere il *primo* simbolo 1 come possibile inizio della sottostringa 101. Nell'automa non deterministico originale, invece, il ramo che rimane nello stato q_0 permette di ignorare il primo 1 e di scegliere il secondo 1 come inizio della sottostringa, accettando così correttamente la parola.

In vista, dell'obiettivo di mostrare che l'insieme $X_1 \cdot X_2$ ottenuto a partire da due insiemi X_1 e X_2 DFA-decidibili è lui stesso DFA-decidibile dobbiamo poter costruire un automa che “concatena” la computazione di un automa con quella del secondo, a questo fine è comodo utilizzare una ulteriore variante della definizione di NDFA che può effettuare transizioni senza fare avanzare il puntatore alla posizione corrente:

Definition 5.6 (Automa Finito non-deterministico con ϵ -mosse). Siano A e Q insiemi finiti, un automa finito non-deterministico con ϵ -mosse di alfabeto A e insieme degli stati Q è una tripla (T, q_0, F) dove

$$T : Q \times A \cup \{\epsilon\} \rightarrow 2^Q$$

$$F \subset Q$$

e $q_0 \in Q$ è uno stato particolare, detto *stato iniziale*.

Per la descrizione della computazione dovremo introdurre una variante della funzione di transizione che prenda in considerazione le ϵ -mosse, a questo scopo introduciamo la seguente notazione per $S \in 2^Q$:

$$T^k(S, a) := \begin{cases} S & \text{se } k = 0 \\ T(T^{k-1}(S, a), a) & k > 0 \end{cases}$$

Definition 5.7 (Algoritmo associato ad un automa finito non-deterministico con ϵ -mosse). L'algoritmo associato ad un automa non-deterministico di alfabeto A e insieme degli stati Q viene descritto definendo le tre funzioni:

- la **funzione di ingresso** che associa ad una parola $w \in A^*$, la configurazione

$$\varepsilon(w) = \left(w, 1, \bigcup_{k \geq 0} T^k(\{q_0\}, \epsilon) \right),$$

- la **funzione di transizione** $\tau : \text{Conf}_{ND}^{A,Q} \rightarrow \text{Conf}_{ND}^{A,Q}$ è data da

$$\tau((w, p, S)) = \left(w, p + 1, \bigcup_{k \geq 0} T^k(T(S, w(p)), \epsilon) \right),$$

- la **funzione di uscita** $\delta : \text{Conf}_{ND}^{A,Q} \rightarrow \{0, 1\}$, data da

$$\delta((w, p, S)) = \begin{cases} 0 & \text{se } p > |w| \text{ e } F \cap S = \emptyset \\ 1 & \text{se } p > |w| \text{ e } F \cap S \neq \emptyset \end{cases}$$

Example 5.8 (Un esempio di ϵ -NDFA). Consideriamo l'alfabeto

$$A = \{0, 1\},$$

l'insieme degli stati

$$Q = \{q_0, q_1, q_2, q_3\},$$

e definiamo il seguente ϵ -N DFA

$$(T, q_0, F),$$

dove

$$F = \{q_3\}$$

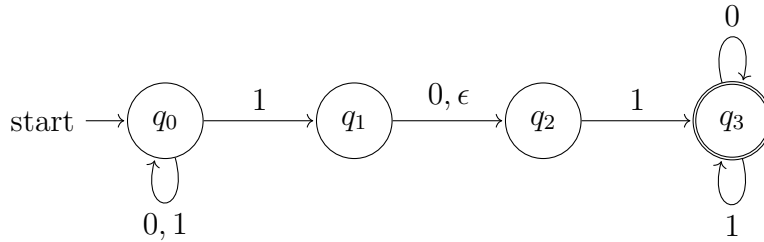
e

$$T : Q \times (A \cup \{\epsilon\}) \rightarrow 2^Q$$

è definita da

$$\begin{aligned} T(q_0, 0) &= \{q_0\}, \\ T(q_0, 1) &= \{q_0, q_1\}, \\ T(q_1, 0) &= \{q_2\}, \\ T(q_1, 1) &= \emptyset, \\ T(q_1, \epsilon) &= \{q_2\}, \\ T(q_2, 0) &= \emptyset, \\ T(q_2, 1) &= \{q_3\}, \\ T(q_3, 0) &= \{q_3\}, \\ T(q_3, 1) &= \{q_3\}. \end{aligned}$$

L'automa può essere rappresentato graficamente come



Per descrivere la computazione consideriamo la funzione di transizione estesa

$$\hat{T} : 2^Q \times A^* \rightarrow 2^Q$$

definita considerando le ϵ -mosse come transizioni gratuite. In particolare, per ogni insieme di stati $S \subseteq Q$ poniamo

$$T^0(S, \epsilon) = S,$$

e

$$T^k(S, \epsilon) = T(T^{k-1}(S, \epsilon), \epsilon)$$

per $k > 0$. La computazione dell'automa consiste quindi nel considerare tutti gli stati ottenibili dopo un numero finito di ϵ -mosse.

- Consideriamo la parola

$$w = 101.$$

Partendo dallo stato iniziale otteniamo

$$\hat{T}(q_0, \epsilon) = \{q_0\}.$$

Dopo aver letto il primo simbolo 1:

$$T(\{q_0\}, 1) = \{q_0, q_1\}.$$

Applicando successivamente le ϵ -mosse:

$$T^0(\{q_0, q_1\}, \epsilon) = \{q_0, q_1\},$$

$$T^1(\{q_0, q_1\}, \epsilon) = T(\{q_0, q_1\}, \epsilon) = \{q_2\}.$$

Quindi dopo il primo simbolo:

$$\widehat{T}(q_0, 1) = \{q_0, q_1, q_2\}.$$

Ora leggiamo il simbolo 0:

$$T(\{q_0, q_1, q_2\}, 0) = \{q_0\} \cup \{q_2\} \cup \emptyset = \{q_0, q_2\}.$$

Non sono possibili ulteriori ϵ -mosse, quindi:

$$\widehat{T}(q_0, 10) = \{q_0, q_2\}.$$

Infine leggiamo 1:

$$T(\{q_0, q_2\}, 1) = \{q_0, q_1\} \cup \{q_3\} = \{q_0, q_1, q_3\}.$$

Poiché

$$q_3 \in \widehat{T}(q_0, 101),$$

la parola 101 è accettata.

- Consideriamo la parola

$$w = 10.$$

Dopo il simbolo 1 abbiamo già visto che

$$\widehat{T}(q_0, 1) = \{q_0, q_1, q_2\}.$$

Leggendo 0 otteniamo:

$$T(\{q_0, q_1, q_2\}, 0) = \{q_0\} \cup \{q_2\} \cup \emptyset = \{q_0, q_2\}.$$

Non essendo presente lo stato finale:

$$q_3 \notin \widehat{T}(q_0, 10),$$

la parola 10 è rifiutata.

Con la nozione appena introdotta è quindi possibile dimostrare in modo diretto che dati due insiemi NDFA-decidibili, l'insieme ottenuto come concatenazione è pure NDFA-decidibile, ovvero la Proposizione 4.3 nel caso non-deterministico, e infatti

Proposition 5.9. Siano X_1 e X_2 decidibili per automa finito non-deterministico allora anche $X_1 \cdot X_2$ è decidibile per automa finito non-deterministico.

Proof. Siano i due automi che decidono $X_1 \subset A^*$ e $X_2 \subset A^*$ rispettivamente dati su insiemi degli stati disgiunti Q_1 e Q_2 , ovvero con $Q_1 \cap Q_2 = \emptyset$: $\mathcal{A}_1 = (T', q'_0, F')$ e $\mathcal{A}_2 = (T'', q''_0, F'')$.

Costruiamo l'automata che decide $X_1 \cdot X_2$ aggiungendo alle transizioni di $\mathcal{A}_1$ e $\mathcal{A}_2$ quelle che vanno dagli stati accettanti di $\mathcal{A}_1$ allo stato iniziale q''_0 del secondo automa, ovvero definiamo l'automata $\mathcal{A}$ con insieme degli stati $Q := Q_1 \cup Q_2$ dove lo stato iniziale è $q_0 := q'_0 \in Q_1$ e gli stati finali accettanti sono definiti come quelli di $\mathcal{A}_2$, dunque:

$$\mathcal{A} := (T, q_0, F)$$

dove

$$T(q, a) := \begin{cases} T'(q, a) & \text{se } q \in F' \text{ e } a \neq \epsilon \text{ oppure } q \notin F', \\ T'(q, a) \cup \{q''_0\} & q \in F' \text{ e } a = \epsilon, \\ T''(q, a) & q \in Q_2, \end{cases}$$

e l'insieme degli stati finali accettanti è dato da $F := F''$. □

Proposition 5.10. Sia dato un insieme $X \subseteq A^*$ per un certo alfabeto finito A . Allora

$$X \text{ è DFA-decidibile} \iff X \text{ è NDFA-decidibile} \iff X \text{ è } \epsilon\text{-NDFA-decidibile}.$$

Proof. Procediamo dimostrando le tre implicazioni.

(DFA $\Rightarrow$ NDFA) Sia (T, q_0, F) un DFA. Basta considerare la funzione di transizione come funzione a valori nell'insieme delle parti:

$$T' : Q \times A \longrightarrow 2^Q,$$

definita da

$$T'(q, a) := \{T(q, a)\}.$$

L'automata così ottenuto è un NDFA che produce esattamente la stessa computazione.

NDFA $\Rightarrow$ ϵ -NDFA) Sia (T, q_0, F) un NDFA. Definiamo

$$T' : Q \times (A \cup \{\epsilon\}) \longrightarrow 2^Q$$

ponendo

$$T'(q, a) := T(q, a), \quad a \in A,$$

e

$$T'(q, \epsilon) := \emptyset.$$

In questo modo l'automata non utilizza mai ϵ -mosse e coincide con quello di partenza.

(ϵ -N DFA $\Rightarrow$ DFA) Sia (T, q_0, F) un ϵ -N DFA.

Consideriamo come insieme degli stati

$$Q' = 2^Q.$$

La funzione di transizione del DFA è definita da

$$T' : 2^Q \times A \longrightarrow 2^Q,$$

ponendo

$$T'(S, a) := \bigcup_{k \geq 0} T^k(T(S, a), \epsilon).$$

Lo stato iniziale è

$$q'_0 := \bigcup_{k \geq 0} T^k(\{q_0\}, \epsilon),$$

mentre l'insieme degli stati finali è

$$F' := \{ S \subseteq Q \mid S \cap F \neq \emptyset \}.$$

È facile controllare che per ogni parola ω le computazioni ottenute tramite i rispettivi algoritmi coincidono, pertanto le decisioni dei due automi coincideranno per ogni ω e quindi decideranno lo stesso insieme.

Ne segue che le tre nozioni di decidibilità coincidono. □

6 Espressioni regolari

Avendo individuato i costrutti rispetto ai quali l'insieme di tutti gli insiemi regolari resta invariato (concatenazione, unione e operatore $*$ di Kleene), si può formalizzare una grammatica data dalla rappresentazione simbolica di alcuni insiemi regolari di base.

Definition 6.1. Fissiamo un alfabeto A . L'insieme $\mathcal{E}$ (anche indicato con E) delle *espressioni regolari* sull'alfabeto A è il più piccolo insieme tale che:

- $\emptyset, \mathbb{1} \in \mathcal{E}$;
- per ogni $a \in A$ $a \in \mathcal{E}$;
- se $E_1, E_2 \in \mathcal{E}$ allora $E_1 + E_2, E_1 \cdot E_2, E_1^* \in \mathcal{E}$

Example 6.2. Alcune espressioni regolari sull'alfabeto $A = 0, 1$ sono:

- $\emptyset$;
- $\emptyset + \emptyset$;
- $(\emptyset + 1)^*$;
- $0 \cdot (\emptyset + 1 \cdot 1)^* \cdot \mathbb{1}$.

Remark 6.3. Chiaramente, fino ad ora, abbiamo usato i simboli $+$, $\cdot$, $*$ in due contesti diversi: per la definizione formale di espressione regolare e per le definizioni di operazioni sui sottoinsiemi degli alfabeti. Questo non è un problema visto che si capisce a quale definizione si fa riferimento in base al contesto.

Definition 6.4 (Interpretazione di un'espressione regolare). Per ogni espressione regolare $e \in E$, di alfabeto A , definiamo un'interpretazione

$$\llbracket \cdot \rrbracket : E \longrightarrow 2^{A^*}$$

dove

$$\llbracket e \rrbracket := \begin{cases} \emptyset & \text{se } e = \emptyset & \text{(insieme vuoto)} \\ \{\epsilon\} & \text{se } e = \mathbb{1} & \text{(insieme singleton parola vuota)} \\ \{a\} & \text{se } e = a \in A & \text{(insieme singleton simbolo)} \\ \llbracket e_1 \rrbracket \cdot \llbracket e_2 \rrbracket & \text{se } e = e_1 \cdot e_2 & \text{(concatenazione)} \\ \llbracket e_1 \rrbracket + \llbracket e_2 \rrbracket & \text{se } e = e_1 + e_2 & \text{(unione)} \\ \llbracket e_1 \rrbracket^* & \text{se } e = e_1^* & \text{(insieme star)} \end{cases}$$

Remark 6.5. Chiaramente, riprendendo la precedente osservazione, con $\llbracket e_1 \rrbracket \cdot \llbracket e_2 \rrbracket$ indichiamo la concatenazione degli insiemi $\llbracket e_1 \rrbracket$ e $\llbracket e_2 \rrbracket$.

Proposition 6.6. Data un'espressione regolare $e \in E$ l'insieme $\llbracket e \rrbracket$ è un insieme regolare.

Proof. Questo risultato è conseguenza delle proprietà di chiusura (per prodotto, somma e star) soddisfatte dagli insiemi DFA-decidibili, e dal fatto che le tre espressioni regolari di base si interpretano con insiemi DFA-decidibili: infatti l'insieme vuoto è deciso dall'automa ... (pagina 31). $\square$

Quello che invece vorremmo mostrare ora è che vale anche l'implicazione inversa della proposizione precedente. Per farlo introdurremo una nuova versione di automa.

Definition 6.7. Un *automa a stati finiti non-deterministico generalizzato* di alfabeto A e insieme degli stati Q è una tripla:

$$(T, q_0, F)$$

dove

$$T : Q \times E \rightarrow 2^Q,$$

$q_0 \in Q$ è detto stato iniziale, $F \subset Q$ è detto insieme degli stati finali accettanti.

Da finire ...

7 Macchin di Turing

Alla luce di quanto fatto prima, sembrerebbe naturale chiamare algoritmica qualsiasi procedura costruita a partire da un automa finito. In particolare è chiaro che una tale procedura può essere descritta in un linguaggio di programmazione e implementata in un qualsiasi sistema informatico. L'affermazione reciproca al contrario sembrerebbe troppo restrittiva: non sembra difendibile che ogni procedura che si voglia definire algoritmica debba essere realizzata mediante il modello di algoritmi associato agli automi finiti (più avanti vedremo degli esempi). Abolire completamente il carattere finito dell'insieme degli stati svuoterebbe di significato la nozione stessa di algoritmo. Un algoritmo è, per definizione, una procedura descritta da un insieme finito di istruzioni/regole — è proprio questo che lo rende "eseguibile", trascrivibile, comunicabile, implementabile in un programma. Se l'insieme degli stati fosse infinito, la "macchina" non sarebbe più descrivibile con una quantità finita di dati. Per conservare il carattere finitario auspicato senza rinunciare a fondamentali capacità di calcolo, si può prescindere dal principio che l'automa non modifica la parola iniziale, dunque nel leggere un certo simbolo dalla memoria potrà successivamente utilizzare questa informazione per modificare il proprio stato ma anche per modificare la memoria scrivendo sulla stessa un nuovo carattere. Questa variante risulta interessante solo se a sua volta lo scorrimento automatico dei caratteri in lettura viene modificato per permettere alla macchina di riesaminare i caratteri che sono stati modificati nei passi di calcolo precedenti, altrimenti la modifica che permette di scrivere non avrebbe alcun impatto sul calcolo effettuato.

Definition 7.1. Una *configurazione ad un nastro di alfabeto A e insieme degli stati Q* è una tripla (f, p, q) , dove

1. il *nastro* è una funzione $f : \mathbb{Z} \rightarrow A'$ dove $A' := A \cup \{\square\}$ (a volte indicato anche con $\tilde{A}$) e tale che $|f^{-1}(A)| < \infty$,
2. $p \in \mathbb{Z}$,

3. $q \in Q'$ dove $Q' = Q \cup \{q_F^0, q_F^1\}$ essendo q_F^0 e q_F^1 due stati che non appartengono a Q .

L'insieme delle configurazioni per un nastro di alfabeto A e insieme degli stati Q sarà denotato con

$$Conf_1^{A,Q} := \{(f, p, q) \text{ che verificano i punti (1), (2), e (3)}\}$$

Il criterio di arresto, che questa volta non è determinato dalla fine della lettura del nastro, deve essere specificato esplicitamente utilizzando un certo numero di stati terminali, che nella definizione sopra sono q_F^0 e q_F^1 . Bisogna inoltre fissare uno stato particolare da utilizzare per la codifica della configurazione iniziale a partire dalla parola di ingresso. Questo stato iniziale indicato da $q_0 \in Q$ determina allora la configurazione iniziale tramite una funzione di input $\epsilon()$. Lo stato iniziale sarà d'ora in poi indicato da q_0 .

Definition 7.2 (Macchina di Turing). Dato un alfabeto A e un insieme finito di stati Q , la *macchina di Turing di alfabeto A e stati Q* è un'applicazione

$$\mu : Q \times \tilde{A} \rightarrow Q' \times \tilde{A} \times \{+1, -1\}.$$

In realtà vorremmo che la macchina di Turing sia allo stesso tempo i seguenti tre oggetti:

$$Q \times A \rightarrow A, \quad Q \times A \rightarrow \{-1, +1\}, \quad Q \times A \rightarrow Q$$

Le precedenti funzioni rappresentano rispettivamente il sovrascrittura di un carattere, il cambio di stato e lo spostamento del puntatore. La funzione μ della definizione precedente formalizza in un'unica funzione i tre precedenti dati, infatti

$$\mu : Q \times \tilde{A} \rightarrow Q' \times \tilde{A} \times \{+1, -1\}$$

prende in input una coppia (q, a) , dove:

- $q \in Q$ è lo *stato corrente* della macchina,
- $a \in \tilde{A}$ è il *carattere letto* sotto il puntatore (può essere un simbolo di A oppure il blank $\square$),

e restituisce una tripla (q', a', m) , dove:

- $q' \in Q'$ è il *nuovo stato* in cui passa la macchina (può essere uno stato “di lavoro” in Q , oppure uno dei due stati terminali q_F^0, q_F^1 se il calcolo deve fermarsi),
- $a' \in \tilde{A}$ è il *carattere da scrivere* al posto di a nella cella corrente,
- $m \in \{+1, -1\}$ è lo *spostamento* del puntatore: $+1$ a destra, -1 a sinistra.

In altre parole, μ è la *tabella delle regole*: “se sono nello stato q e leggo il carattere a , allora scrivo a' , mi sposto di m e passo allo stato q' ”.

Definition 7.3. Sia μ una macchina di Turing di alfabeto A e insieme degli stati Q , l'*algoritmo formale* $\mathcal{A}_\mu$ associato a μ è la tupla $(A, \{0, 1\}, Conf_1^{A,Q}, \epsilon, \tau, \delta)$: dove

1. l'insieme delle configurazioni è dato da

$$Conf_1^{A,Q}$$

2. la funzione di input $\epsilon : A^* \rightarrow Conf_1^{A,Q}$ dove $\epsilon(w) = (f_w, 1, q_0)$ con f_w corrisponde alla funzione nastro per cui

$$f_w(n) = \begin{cases} w(n) & \text{se } 1 \leq n \leq |w| \\ \square & \text{altrimenti} \end{cases}$$

3. la funzione di transizione

$$\tau : Conf_1^{A,Q} \rightarrow Conf_1^{A,Q}$$

è definita come $\tau((f, p, q)) = (f', p', q')$ dove supposta $\mu(q, f(p)) = (q', a', m)$ si porrà $p' = p + m$ e

$$f'(x) = \begin{cases} f(x) & \text{se } x \neq p \\ a' & \text{se } x = p. \end{cases}$$

4. la funzione di uscita δ associa ad una configurazione data (f, p, q) dove $q \in \{q_F^0, q_F^1\}$ il valore 0 se $q = q_F^0$ e il valore 1 se $q = q_F^1$.

Cerchiamo di capire come si esegue un passo di calcolo in questo caso.

Remark 7.4. Data una configurazione $(f, p, q) \in Conf_1^{A,Q}$ (nastro, posizione del puntatore, stato corrente), un passo di calcolo si esegue così:

1. si legge il carattere sotto il puntatore: $a := f(p)$;
2. si applica μ : $\mu(q, a) = (q', a', m)$;
3. si aggiorna lo stato: il nuovo stato è q' ;
4. si aggiorna la posizione: $p' = p + m$ (il puntatore si sposta di una cella a destra o sinistra);
5. si aggiorna il nastro: resta identico ovunque tranne che nella cella p , dove viene scritto a' :

$$f'(x) = \begin{cases} f(x) & x \neq p \\ a' & x = p. \end{cases}$$

Questo definisce la funzione di transizione $\tau : Conf_1^{A,Q} \rightarrow Conf_1^{A,Q}$, $\tau(f, p, q) = (f', p', q')$.

Vediamo ora l'avvio e l'arresto del calcolo.

Remark 7.5.

- *Avvio*: data una parola di input $w \in A^*$, la configurazione iniziale è $\epsilon(w) = (f_w, 1, q_0)$: il nastro contiene w scritta a partire dalla cella 1 (blank altrove), il puntatore parte dalla cella 1 (inizio della parola), lo stato iniziale è q_0 .

- *Iterazione*: si applica ripetutamente τ per ottenere la sequenza di configurazioni

$$\epsilon(w), \tau(\epsilon(w)), \tau^2(\epsilon(w)), \dots$$

questa è l'esecuzione del calcolo, passo dopo passo.

- *Arresto*: il calcolo si ferma quando si raggiunge una configurazione il cui stato è q_F^0 oppure q_F^1 (questi sono gli unici stati per cui μ non è più definita/rilevante, essendo terminali).
- *Output*: se ci si ferma in q_F^1 l'output è 1 (accetta); se ci si ferma in q_F^0 l'output è 0 (rifiuta). Se non ci si ferma mai, l'algoritmo non produce output su quell'input (il calcolo diverge).

In sintesi, μ è la “regola locale” (cosa fare dato stato+carattere letto); τ applica questa regola a una configurazione globale producendone la successiva; ϵ dice come tradurre l'input iniziale in una configurazione; l'iterazione di τ a partire da $\epsilon(w)$, fino a un eventuale stato terminale, è l'esecuzione dell'algoritmo $\mathcal{A}_\mu$ sull'input w .

Definition 7.6. Sia $X \subset A^*$, allora X si dice *decidibile tramite Macchina di Turing* (o *Turing-decidibile*) se esiste Q insieme finito di stati e una macchina di alfabeto A e insieme degli stati Q per cui l'algoritmo associato $\mathcal{A}_\mu$ decide l'insieme X .

Analogamente si ottiene la definizione di insieme *semi-decidibile* per macchina di Turing se esiste una macchina di Turing tale che l'algoritmo associato semi-decide X .

Proposition 7.7. Ogni insieme decidibile per macchina di Turing è anche semidecidibile per macchina di Turing.

Proof. Per dimostrare questa proposizione è sufficiente costruire una macchina $\tilde{\mu}$ che per ogni parola $w \in A^*$ e $w \notin X$ entri in una configurazione non terminale. A questo scopo, sia μ la macchina che decide X e sia Q il suo insieme degli stati, andiamo a modificarla nelle transizioni che coinvolgono il suo stato finale non accettante. Consideriamo la macchina μ^* di alfabeto A e insieme degli stati $Q^* = Q \cup \{q^*\}$ che in corrispondenza delle transizioni di μ che coinvolgono lo stato non accettante opera una transizione verso una configurazione (f, p, q^*) per cui

$$\tau(f, p, q^*) = (f, p', q^*)$$

che porta la macchina in una computazione che non termina. Più precisamente, data la macchina

$$\mu : Q \times A' \longrightarrow Q' \times A' \times \{-1, +1\},$$

si costruisce la macchina

$$\mu^* : Q^* \times A' \longrightarrow Q^{*'} \times A' \times \{-1, +1\},$$

dove

$$Q^* = Q \cup \{q^*\}$$

e

$$\mu^*(q, a) = \begin{cases} \mu(q, a) & \text{se } \mu(q, a) = (q', a', m) \text{ e } q' \neq q_F^0, \\ (q^*, a, +1) & \text{altrimenti.} \end{cases}$$

□

8 Teoria della complessità

Definition 8.1. Data una macchina di Turing μ di alfabeto A e insieme degli stati Q

$$\mu : Q \times A' \rightarrow Q' \times A' \times \{-1, +1\}$$

con $A' = A \cup \{\square\}$, $Q' = Q \cup \{q_F^0, q_F^1\}$:

1. sia

$$C = (c_0, \dots, c_n)$$

una computazione di μ , il *costo in tempo* di C è dato dalla lunghezza di C diminuita di 1;

2. il *costo in spazio* è dato dalla cardinalità dell'insieme di interi p tali che almeno per una delle configurazioni $c_i = (f_i, p_i, q_i)$ si ha $p = p_i$:

$$|\{p \mid (f, p, q) \in C\}|;$$

3. si dirà che μ si arresta in tempo t (rispettivamente spazio s) sull'input w , se C è la computazione di μ a partire dalla configurazione iniziale associata a w , C raggiunge una configurazione terminale e il costo in tempo di C è t (rispettivamente il costo in spazio è s); in questo caso si userà la notazione $T_\mu(w)$ (rispettivamente $S_\mu(w)$) per indicare il tempo di arresto t (rispettivamente lo spazio di arresto s).

Di solito ci si interessa al comportamento nel caso peggiore:

$$T_{max}(n) := \max_{w \in A^*, |w| \leq n} T_\mu(w)$$

Si noti che la definizione di costo in tempo di una computazione associata ad una parola da una macchina di Turing si basa sulla lunghezza della computazione e pertanto si adatta a qualsiasi algoritmo formale (non necessariamente per una macchina di Turing). Viceversa, per quanto riguarda la nozione di costo in spazio la definizione è basata sulla specifica configurazione e pertanto non necessariamente è trasportabile nel caso di altri algoritmi formali.

Definition 8.2. Sia μ una macchina di Turing sull'alfabeto A e con insieme di stati Q , e siano T , ed S due funzioni

$$T : \mathbb{N} \rightarrow \mathbb{N},$$

$$S : \mathbb{N} \rightarrow \mathbb{N}.$$

Si dice che μ si *arresta in tempo* T (rispettivamente *in spazio* S) se per ogni $w \in A^*$ di lunghezza al più uguale ad n la macchina si arresta sull'input w in tempo (risp. spazio) al più uguale a $T(n)$ (risp. $S(n)$).

Definition 8.3. Siano T e S funzioni come nella definizione sopra.

1. La macchina μ di alfabeto A *decide l'insieme* X *in tempo* T (risp. *spazio* S) se μ decide X e se esiste un intero N tale che per ogni intero $n \geq N$ e per ogni parola $w \in A^*$ di lunghezza al più uguale a n la macchina μ si ferma a partire da w in tempo al più uguale a $T(n)$ (risp. spazio al più uguale ad $S(n)$).

2. La *classe di complessità* $\text{DTIME}_1^A(T)$ (risp. $\text{DSpace}_1^A(S)$) è l'insieme che contiene gli insiemi $X \subset A^*$ per i quali esiste almeno una macchina di Turing di alfabeto A che decide X in tempo T (risp. in spazio S).

Con abuso di notazione, spesso scriveremo $T(n)$ invece che T .

Remark 8.4. L'indice 1 in basso utilizzato in questa notazione fa riferimento al fatto che nella macchina viene utilizzato un solo nastro. Con questa notazione si può notare che l'insieme X degli interi divisibili per 9 (come pure tutti gli insiemi decidibili per automa finito) appartengono alle classi $\text{DTIME}_1^{\{0,1,\dots,9\}}(n)$ e $\text{DSpace}_1^{\{0,1,\dots,9\}}(n)$.

È anche possibile interessarsi al caso medio, allora si prenderebbe in considerazione non la funzione T_{max} ma una media su tutti i possibili input

$$T_{avg}(n) := \frac{\sum_{w \in A^N} T_\mu(w)}{|A^N|}$$

dove $A^N = \{w \in A^* \mid |w| \leq N\}$.

Definition 8.5. Una funzione

$$T : \mathbb{N} \rightarrow \mathbb{N}$$

viene detta *funzione di complessità* se è una funzione non decrescente e se $T(n) \geq n$ definitivamente.

8.1 Look-up Table

Il lemma seguente mostra che il valore puntuale delle funzioni di complessità non corrisponde alla reale difficoltà del problema, in quanto, data una macchina di Turing che decide un problema X con un certo tempo di arresto, è sempre possibile trovarne un'altra che decide lo stesso problema con complessità lineare per tutte le parole di input fino ad una lunghezza fissata, dunque la nozione di complessità ha senso come misura della difficoltà di un problema solo quando si considera il comportamento asintotico dell'algoritmo.

Lemma 8.6 (Look-up Table). Dato un insieme $X \subset A^*$, supponiamo che esista una macchina di Turing μ che lo decide in tempo $T(n)$ e spazio $S(n)$. Per un intero qualsiasi N siano

$$T'(n) = \begin{cases} n & n < N \\ T(n) + 2N & \text{altrimenti} \end{cases}$$

e sia

$$S'(n) = \begin{cases} n & n < N \\ S(n) & \text{altrimenti} \end{cases}$$

allora esiste una macchina di Turing μ' che decide X in tempo $T'(n)$ e spazio $S'(n)$.

Proof. Si può modificare la macchina μ per includere nell'insieme degli stati direttamente la risposta che corrisponde ad una data parola di input (in un numero finito di casi).

Ed è quello che faremo nel caso delle parole di lunghezza inferiore uguale ad N , in questo modo la nuova macchina si fermerà non appena sarà stata letta la parola di input. In questo caso la macchina dovrà leggere il nastro e quindi si arresterà in tempo al più N . In caso fossimo in presenza di una parola più lunga bisognerebbe riprendere la computazione della macchina M , reimpostando la macchina nello stato iniziale di M e nella posizione iniziale.

Sia

$$Q' = Q \cup \{q_w \mid w \in A^N\} \cup \{q_R\}$$

dove lo stato q_R sarà utilizzato come lo stato utilizzato per il ritorno alla casella iniziale nel caso in cui l'input si trovi più lungo di N simboli.

Dunque la macchina di Turing μ' è definita a partire dalla macchina μ :

$$\mu'(q, a) = \begin{cases} \mu(a, q) & \text{se } q \in Q, \\ (q_{wa}, a, +1) & \text{se } q = q_w \text{ per qualche } w \in A(m) \text{ con } m \leq N \text{ e } |wa| \leq N, \\ (q_R, a, +1) & \text{se } q = q_w \text{ per qualche } w \in A(m) \text{ con } a \neq \square \text{ e } |w| = N, \\ (q_R, a, -1) & \text{se } q = q_R \text{ e } a \neq \square, \\ (q_0, a, +1) & \text{se } q = q_R \text{ e } a = \square, \\ (q_F^1, a, +1) & \text{se } q = q_w \text{ per qualche } w \in A(m) \text{ con } m \leq N, a = \square \text{ e } w \in X, \\ (q_F^0, a, +1) & \text{se } q = q_w \text{ per qualche } w \in A(m) \text{ con } m \leq N, a = \square \text{ e } w \notin X; \end{cases}$$

Lo stato iniziale q'_0 di μ' coincide con lo stato associato alla parola vuota q_ϵ mentre lo stato q_0 di μ deve essere considerato come uno stato generico di μ' .

Per ogni parola di lunghezza inferiore o uguale ad N la computazione viene effettuata in modo simile alla computazione per automa finito, per le altre parole la macchina μ' compie una lettura ed un ritorno alla posizione iniziale che non altera l'input e dunque quando la macchina torna alla posizione iniziale e passa nello stato iniziale (di μ) la computazione si svolge esattamente come per μ . $\square$

Remark 8.7. Il significato di questo lemma è che ha senso analizzare la complessità (dell'algoritmo di decisione associato), soltanto relativamente ad un insieme X infinito, poiché nel caso di un X finito si può sempre decidere il problema in un numero di passi finito, pari alla lunghezza della sua codifica sull'alfabeto (associando alla parola uno stato “ad hoc” e facendo una transizione accettante in base alla parola). Inoltre, poiché è possibile decidere il complementare di un insieme decidibile (invertendo nella tabella lo stato accettante q_F^1 e quello non accettante q_F^0), in pratica la questione della decidibilità si pone solo quando X è infinito e co-infinito, mentre è banale quando l'insieme è finito oppure co-finito.

Remark 8.8. Il modello di calcolo “macchina di Turing” è più potente del modello “automa a stati finiti”, in quanto per ogni automa (T, q_0, F) resta definita una macchina di Turing μ sullo stesso alfabeto e sullo stesso insieme di stati per cui la computazione dell'algoritmo associato alla macchina μ coincide con l'algoritmo associato all'automa, più precisamente è

sufficiente prendere

$$\mu(q, a) := \begin{cases} (T(q, a), a, +1) & a \neq \square \\ (q_F^1, \square, +1) & \text{se } q \in F \\ (q_F^0, \square, +1) & \text{se } q \notin F \end{cases}$$

Ogni macchina di Turing associata ad un automa finito si arresta in tempo e spazio $T(n) = n$.

Proposition 8.9. Sia $T : \mathbb{N} \rightarrow \mathbb{N}$ una funzione di complessità le inclusioni seguenti sono sempre vere.

$$\text{DTIME}_1^A(T) \subset \text{DSpace}_1^A(T) \subset \bigcup_{\epsilon > 0} \text{DTIME}_1^A(2^{\epsilon T}).$$

Proof. Fissato l'alfabeto $A = \{0, 1\}$, sia $X \in \text{DTIME}_1^A(T)$. Poiché ad ogni passo di computazione è accessibile una sola posizione allora la massima posizione raggiungibile in $T(n)$ passi di computazione non può essere maggiore di $T(n)$.

Sia ora μ una macchina di Turing di alfabeto A ed insieme degli stati Q , e sia $k = |A|$ e $d = |Q|$.

Per ogni intero s esistono $(2s + 1)k^{2s+1}d$ configurazioni distinte tali che coinvolgono durante la computazione soltanto le posizioni di indice compreso tra $1 - s$ e $1 + s$. Ora se il costo in spazio della computazione di μ a partire da una parola di input w è maggiorato da s (con s più grande della lunghezza $|w|$) allora tutte le configurazioni coinvolgono posizioni di indice compreso tra $1 - s$ e $1 + s$. Quindi se la lunghezza della computazione fosse e solo se maggiore di $(2s + 1)k^{2s+1}d$ si avrebbe necessariamente una configurazione ripetuta e perciò l'insieme delle configurazioni di μ a partire da w sarebbe periodico, si avrebbe non terminazione, contraddicendo il fatto che nel caso peggiore la computazione termina in un tempo finito (dalla definizione di decidibilità). Quindi il tempo di arresto è certamente maggiorato da $(2s + 1)k^{2s+1}d$ che per una costante c abbastanza grande indipendente da s si ha che il tempo di arresto è maggiorato da 2^{cs} . $\square$

9 Prerequisiti

Per tutto il documento, la definizione di funzione sarà quella di funzione parziale. Pertanto, data una funzione $f : A \rightarrow B$, indicheremo con $\text{dom}(f)$ il dominio della funzione f . Inoltre, se $x \in A \setminus \text{dom}(f)$ (quindi f non è definita in x), allora scriveremo che $f(x) := \perp$. Inoltre, quando scriveremo *alfabeto* intenderemo insieme. Useremo la notazione $f^0 := \text{Id}_x$ per ogni funzione $f : X \rightarrow X$.

Definition 9.1 (Funzione caratteristica). La funzione $\mathbf{1}_X$ è la *funzione caratteristica* (o indicatrice) dell'insieme X , definita come

$$\mathbf{1}_X(w) = \begin{cases} 1 & w \in X \\ 0 & w \notin X. \end{cases}$$

Definition 9.2 (Partizione).

Definition 9.3 (Grafo).

Definition 9.4 (Matrice adiacenza).

Proposition 9.5. Se M e/' matrice adiacenza per un grafo G , allora $M_{i,j}^k$ e/' il numero di cammini di lunghezza k che connettono v_i e v_j .

Definition 9.6 (Grafo etichettato). Sia A un insieme, detto *alfabeto*. Un *grafo etichettato* su A è una terna

$$G = (V, E, L),$$

dove $E \subset V \times A \times V$ e

- chiamando con $\pi_{1,3}$ la proiezione sulla prima e terza componente, $(V, \pi_{1,3}(E))$ è un grafo orientato;
- $L : E \rightarrow A$ è una funzione, detta *funzione di etichettatura*, che associa ad ogni arco un elemento di A chiamato *etichetta* associata all'arco.

Proposition 9.7. Prop identica a quella dei grafi normali ma con i grafi etichettati.

9.1 Semianelli e serie formali

Per estendere ai grafi etichettati l'interpretazione combinatoria delle potenze successive della matrice di adiacenza, è necessario dotare l'insieme delle etichette di una struttura algebrica con somma e prodotto. Introduciamo qui, in modo autonomo, gli strumenti algebrici necessari: semianelli, matrici a coefficienti in un semianello, serie formali e algebre di Kleene.

Definition 9.8 (Semigruppo). Un *semigruppo* è una coppia $(S, *)$ dove S è un insieme e

$$* : S \times S \rightarrow S$$

è un'operazione binaria (cioè una funzione che ad ogni coppia ordinata $(x, y) \in S \times S$ associa un elemento $x * y \in S$) tale che valga la proprietà associativa:

$$(x * y) * z = x * (y * z) \quad \text{per ogni } x, y, z \in S.$$

Exercise 3. $(\mathbb{N}, +)$ è un semigrupp: la somma di due numeri naturali è un numero naturale ed è associativa. Anche $(\mathbb{N}, \cdot)$ è un semigrupp. Un altro esempio è $(A^*, \cdot)$, l'insieme delle stringhe finite su un alfabeto A con l'operazione di concatenazione, che è associativa ma non commutativa se $|A| > 1$.

Definition 9.9 (Semianello). Un *semianello* è una struttura algebrica

$$(S, +, \cdot)$$

dove S è un insieme e

$$+ : S \times S \rightarrow S, \quad \cdot : S \times S \rightarrow S$$

sono due operazioni binarie su S , dette rispettivamente *somma* e *prodotto*, tali che:

- $(S, +)$ è un semigrupp (Definizione 9.8), cioè $+$ è associativa;
- $(S, \cdot)$ è un semigrupp, cioè $\cdot$ è associativa;
- valgono le leggi di distribuzione tra somma e prodotto:

$$x \cdot (y + z) = (x \cdot y) + (x \cdot z), \quad (y + z) \cdot x = (y \cdot x) + (z \cdot x)$$

per ogni $x, y, z \in S$.

Definition 9.10 (Semianello additivamente commutativo, zero, identità). Sia $(S, +, \cdot)$ un semianello (Definizione 9.9).

- $(S, +, \cdot)$ si dice *additivamente commutativo* se

$$x + y = y + x \quad \text{per ogni } x, y \in S.$$

- Un elemento $0 \in S$ si dice *zero* di $(S, +, \cdot)$ se

$$x + 0 = 0 + x = x \quad \text{e} \quad x \cdot 0 = 0 \cdot x = 0 \quad \text{per ogni } x \in S.$$

- Un elemento $1 \in S$ si dice *identità* (o *unità*) di $(S, +, \cdot)$ se

$$x \cdot 1 = 1 \cdot x = x \quad \text{per ogni } x \in S.$$

Si noti che, quando esistono, lo zero e l'identità di un semianello sono unici: se 0 e $0'$ fossero entrambi zeri, si avrebbe $0 = 0 + 0' = 0'$; analogamente per l'identità.

Definition 9.11 (Semianello idempotente). Un semianello $(S, +, \cdot)$ additivamente commutativo, con zero 0 e identità 1 (Definizione 9.10), si dice *idempotente* se l'operazione di somma è idempotente, ossia

$$x + x = x \quad \text{per ogni } x \in S.$$

Definition 9.12 (Matrici su un semianello). Sia S un insieme munito di struttura di semianello commutativo² con zero 0 e identità 1 , e sia n un intero positivo. Si definisce

$$M_n(S) := \{A : \{1, \dots, n\} \times \{1, \dots, n\} \rightarrow S\}$$

l'insieme di tutte le matrici $n \times n$ a coefficienti in S (cioè le funzioni che ad ogni coppia di indici (i, j) associano un elemento $A_{ij} \in S$), munito delle operazioni:

$$(A + B)_{ij} := A_{ij} + B_{ij} \quad (\text{somma elemento per elemento}),$$

$$(A \cdot B)_{ij} := \sum_{k=1}^n A_{ik} \cdot B_{kj} \quad (\text{prodotto righe per colonne}).$$

Con tali operazioni $(M_n(S), +, \cdot)$ è a sua volta un semianello (le proprietà associative e distributive si verificano direttamente dalle corrispondenti proprietà in S), ed è in particolare un semigrupp additivamente commutativo se S lo è. La matrice $0_n \in M_n(S)$ con tutti gli elementi uguali a 0 è l'elemento neutro per l'operazione additiva, e la matrice identità

$$I_n := \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{pmatrix} \in M_n(S)$$

è l'identità del semianello $M_n(S)$.

Se S contiene più di un elemento ed $n > 1$, allora $M_n(S)$ non può essere moltiplicativamente commutativo (come nel caso classico delle matrici su un anello, è sufficiente esibire due matrici A, B con $AB \neq BA$).

Per $A \in M_n(S)$ e $i, j \in \{1, \dots, n\}$, ricordiamo che A_{ij} denota l'elemento di A alla riga i -esima e alla colonna j -esima.

Definition 9.13 (Trasposta). La *trasposta* di una matrice $A \in M_n(S)$ è la matrice $A^t \in M_n(S)$ definita da

$$(A^t)_{ij} := A_{ji} \quad \text{per ogni } i, j \in \{1, \dots, n\}.$$

Per ogni coppia di matrici $A, B \in M_n(S)$ valgono le identità

$$(A^t)^t = A, \quad (A + B)^t = A^t + B^t, \quad (AB)^t = B^t A^t.$$

Definition 9.14 (Monoide). Un *monoide* è una terna $(M, \cdot, 1)$ dove M è un insieme,

$$\cdot : M \times M \rightarrow M$$

è un'operazione binaria associativa (cioè $(M, \cdot)$ è un semigrupp, Definizione 9.8), e $1 \in M$ è un elemento tale che

$$1 \cdot m = m \cdot 1 = m \quad \text{per ogni } m \in M.$$

²Con *commutativo*, quando non altrimenti specificato, si intende qui additivamente commutativo (Definizione 9.10).

Definizione 9.15 (Serie formali). Dato un monoide $(M, \cdot, 1)$ (Definizione 9.14) e un semi-anello $\mathbb{K}$, si consideri l'insieme delle *serie formali* a coefficienti in $\mathbb{K}$ sul monoide M :

$$\mathbb{K}[M] := \left\{ \sum_{m \in M} \alpha_m m \mid \alpha_m \in \mathbb{K} \text{ per ogni } m \in M \right\},$$

cioè l'insieme delle funzioni $\alpha : M \rightarrow \mathbb{K}$, scritte formalmente come somme $\sum_{m \in M} \alpha_m m$ dove $\alpha_m := \alpha(m)$.

Su $\mathbb{K}[M]$ resta definita un'operazione di somma, elemento per elemento sui coefficienti,

$$\sum_{m \in M} \alpha_m m + \sum_{m \in M} \beta_m m := \sum_{m \in M} (\alpha_m + \beta_m) m,$$

e un'operazione di prodotto (detta *prodotto di convoluzione*)

$$\left(\sum_{m' \in M} \alpha_{m'} m' \right) \left(\sum_{m'' \in M} \beta_{m''} m'' \right) = \sum_{m \in M} \left(\sum_{\substack{m', m'' \in M \\ m' m'' = m}} \alpha_{m'} \beta_{m''} \right) m.$$

Si noti che nella definizione di prodotto tra serie formali compaiono due nozioni distinte di prodotto:

- il prodotto tra coefficienti $\alpha_{m'} \beta_{m''}$, che è il prodotto del semi-anello $\mathbb{K}$ (Definizione 9.9);
- il prodotto tra elementi del monoide $m' m''$, che è il prodotto del monoide M (Definizione 9.14).

Con queste due operazioni, $(\mathbb{K}[M], +, \cdot)$ è a sua volta un semi-anello.

Definizione 9.16 (Ordine indotto da un semi-anello idempotente). Sia $(S, +, \cdot)$ un semi-anello idempotente (Definizione 9.11). Si definisce su S la relazione binaria $\leq \subseteq S \times S$ data da

$$a \leq b \quad \text{se e solo se} \quad a + b = b.$$

Si verifica facilmente che $\leq$ è un ordine parziale, ossia soddisfa:

- *riflessività*: $a \leq a$ per ogni $a \in S$ (segue da $a + a = a$, cioè dall'idempotenza);
- *antisimmetria*: se $a \leq b$ e $b \leq a$ allora $a = b$ (da $a + b = b$ e $b + a = a$, usando la commutatività additiva, si ottiene $a = b + a = a + b = b$);
- *transitività*: se $a \leq b$ e $b \leq c$ allora $a \leq c$ (da $a + b = b$ e $b + c = c$ segue $a + c = a + (b + c) = (a + b) + c = b + c = c$).

Definizione 9.17 (Estremo superiore). Sia $(S, \leq)$ un insieme parzialmente ordinato (ad esempio quello indotto come in Definizione 9.16) e sia $T \subseteq S$. Si dice che $y \in S$ è l'*estremo superiore* di T , e si scrive $y = \sup T$, se e soltanto se:

1. $x \leq y$ per ogni $x \in T$ (cioè y è un maggiorante di T);

2. per ogni $y' \in S$ che verifica la proprietà al punto (1), si ha $y \leq y'$ (cioè y è il più piccolo dei maggioranti).

Per un insieme finito, l'estremo superiore è la somma degli elementi:

$$\sup\{a_1, a_2\} = a_1 + a_2, \quad \sup\{a_1, \dots, a_n\} = a_1 + \dots + a_n.$$

(Questo segue induttivamente dalla definizione di $\leq$: $a_1 \leq a_1 + a_2$ poiché $a_1 + (a_1 + a_2) = (a_1 + a_1) + a_2 = a_1 + a_2$ per idempotenza, e analogamente per a_2 ; la minimalità segue dalla definizione di $+$ come somma nel semianello.)

Si noti che l'estremo superiore di un insieme infinito potrebbe non esistere in generale.

Definition 9.18 (Algebra di Kleene). L'algebra delle serie formali $\mathbb{K}[M]$ (Definizione 9.15) su un semianello idempotente $\mathbb{K}$ si dice *algebra di Kleene* se ad ogni elemento $a \in \mathbb{K}$ è associato un elemento $a^* \in \mathbb{K}$, detto *chiusura di Kleene* di a , tale che

$$a b^* c = \sup_{n \geq 0} a b^n c, \quad \text{dove } b^n = \begin{cases} 1 & \text{se } n = 0 \\ b \cdot b^{n-1} & \text{se } n \geq 1, \end{cases} \quad (1)$$

per ogni $a, b, c \in \mathbb{K}$, con il sup preso rispetto all'ordine parziale di Definizione 9.16.

L'equazione (1) asserisce, in particolare, la proprietà di chiusura mediante l'operatore $*$: postula l'esistenza dell'estremo superiore per famiglie infinite della forma $\{a b^n c\}_{n \geq 0}$, anche quando l'esistenza del sup non è garantita in generale per un semianello idempotente qualsiasi.